Task to Budget Sync: Duplicates and Error Backlog Handling

NiCE account | Workato recipes for billing milestone budget updates and SF case creation

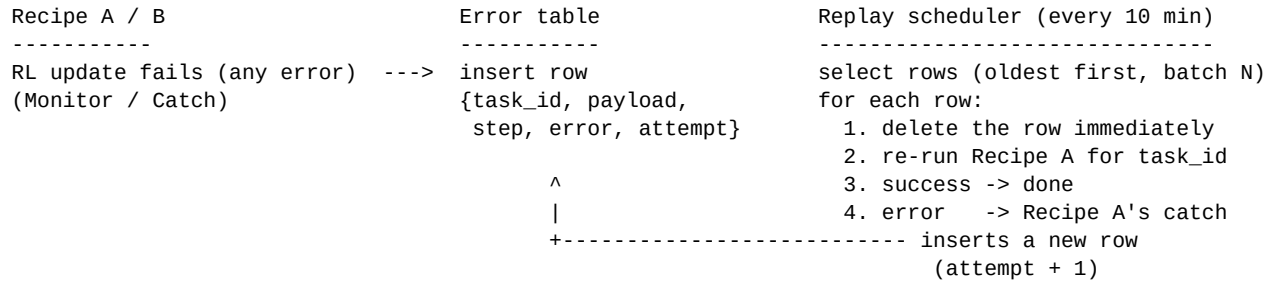
Items marked **(verify)** depend on Workato plan limits or Rocketlane behaviour.

1. Current error backlog handling

- All three recipes run at concurrency 1.
- Error handling is set to **do not retry** for all errors. Any failure goes to the Backlog Error Table.
- Transient errors (429, 5xx) become manual backlog work, the same as permanent data errors.
- Step 33 (Update budgets) runs with wait for response off, so failed budget updates are **not caught or logged** at all.
- If the case ID write-back fails after the SF case is created, the next event creates a duplicate case.

2. Proposed error handling: park, replay, delete

Any failed Rocketlane update is pushed to the error table and processed later by a scheduled recipe. No long in-job retries, so the concurrency 1 queue is never blocked.



Rules

- **Push on any RL update error.** Wrap every RL write in a Monitor block. The Catch inserts a row and the job ends cleanly.
- **Scheduler runs every 10 minutes** and picks up rows oldest first, up to a batch size to stay inside job time limits.
- **Delete the row immediately** when it is picked up. The row's job is to say 'this still needs doing'; once the scheduler owns it, the row goes.
- **If the re-run fails again**, the normal Catch inserts a fresh row, so the task goes back to the backlog with attempt + 1.
- **Re-runs are safe** because Recipe A reads the task and budget fresh and only writes the diff.

Safeguards (recommended additions)

- **Attempt cap (e.g. 5):** without a cap, a permanent error (400, 404, 422) loops every 10 minutes forever. On the 5th failure, insert with status FAILED, which the scheduler ignores, and send one alert.
- **One row per task:** since Recipe A reads fresh state, five rows for the same task do the same work. Before inserting, check for an existing row for the task_id and update it instead.

- **Crash window:** deleting first means a scheduler job that dies between the delete and the re-run loses the row. Wrap the re-run in a Monitor whose Catch re-inserts the row, which covers normal failures. A hard crash or timeout still leaves a small gap; if that matters, delete after the re-run succeeds instead. **(decide)**
- **429 specifically:** the 10 minute spacing already acts as backoff. Add a short Wait between rows so a batch does not trigger the limit again.

Error table schema (Data table task_sync_errors)

Column	Purpose
task_id	Task to re-run (one pending row per task)
recipe, step	Where it failed (A or B, step name)
http_status, error_message	What failed, for triage
payload	Original webhook payload, needed for the re-run
attempt	Starts at 1, incremented on each re-insert
status	PENDING (scheduler picks up) or FAILED (alert, manual)
created_at	Ordering and ageing alerts

What happens by error type

Error	First failure	Replay result
429, 5xx, timeout	Insert row, attempt 1	Usually succeeds on a later run
409/412 (concurrent edit)	Insert row	Fresh read on re-run resolves it
400/404/422 (data error)	Insert row	Fails every time; reaches the attempt cap, becomes FAILED, alert
401/403 (auth)	Insert row	Fails until the connection is fixed; alert early, do not wait for the cap

3. Avoiding duplicates: 3 layers

Re-running is only safe if running twice gives the same result. Three layers guarantee that.

Layer	Where	What it stops
1. Event dedupe	Webhook recipe	Same event delivered twice
2. Diff check	Recipe A	Budget writes when nothing changed
3. Case upsert	Recipe B	Duplicate SF cases after a failed write-back

Layer 1: Event dedupe (webhook recipe)

Build a key from the event, look it up in a Data table, and stop if it was already seen.

```
key = task_id | trigger_type | status | due_date | updatedAt
seen?  -> stop
new?   -> save key (24h TTL), continue
```

#	Arrival	Status	updatedAt	event_id	Result
1	10:00:04	Completed	10:00:04	T-123|status|COMPLETED|10:00:04	New, processed
2	10:00:06	Completed	10:00:04	T-123|status|COMPLETED|10:00:04	**Duplicate**, stopped

#	Arrival	Status	updatedAt	event_id	Result
3	10:05:00	In Progress	10:05:00	T-123|status|IN_PROGRESS|10:05:00	New, processed

Only exact repeats are dropped. A real status change has a different key and always passes. Replays from the error table skip this layer because they are not webhook deliveries.

Layer 2: Diff check (Recipe A)

Two different events (Status updated and Due date changed) fire for the same task and lead to the same budget values.

```
Event 1: target  = {status: COMPLETED, actual_date: 2026-10-12}
         current = {status: IN_PROGRESS, actual_date: null}
         diff    = {status: COMPLETED, actual_date: 2026-10-12}   -> PUT only these

Event 2: target  = {status: COMPLETED, actual_date: 2026-10-12}
         current = {status: COMPLETED, actual_date: 2026-10-12}   <- written by event 1
         diff    = {}                                              -> no PUT
```

```
def diff(target, current):
    return {k: v for k, v in target.items() if current.get(k) != v}
```

Layer 3: Case upsert (Recipe B)

The Case is created, then writing the case ID back to the Rocketlane task fails.

```
OLD:  Event 1: create Case 500-A -> write-back fails
      Event 2: task has no case number -> create Case 500-B      (duplicate)

NEW:  Event 1: upsert on RL_Task_Id__c = T-123 -> creates Case 500-A
               write-back fails -> row pushed to error table
      Replay:  upsert -> Case with T-123 exists -> returns 500-A (no new case)
               write-back succeeds
```

All layers together

Event	Layer 1	Layer 2	Layer 3
Status, 10:00:04	New, pass	Writes status and actual date	Creates Case 500-A
Due date, 10:00:05	New, pass	Nothing changed, no PUT	Case number set, skip
Status redelivery, 10:00:06	**Dropped**	not reached	not reached
Replay of a failed run (10 min later)	not applicable	Diff only writes what is still missing	Upsert returns existing case

Result: one budget write and one case, however many times an event or a replay runs.

4. Old vs new

Situation	Current	Proposed
RL update fails	Not retried; to Backlog Error Table, manual work	Row in error table; replayed every 10 min
Budget update fails (step 33)	Not caught (wait for response off)	Awaited; failure inserts an error row
Same event twice	Runs twice	Dropped at webhook
Nothing changed	Full PUT of all fields	No write
Case write-back fails	Duplicate case on next event	Upsert returns existing case

Situation	Current	Proposed
Permanent data error	Sits in backlog, no limit	Attempt cap, FAILED status, one alert

5. Open items

1. Decide delete-first (as designed) or delete-after-success for the replay row. **(decide)**
2. Attempt cap value and the batch size per scheduler run.
3. Whether the Data table supports a unique constraint on event_id and task_id. **(verify)**
4. Workato job timeout versus batch size, so a run finishes inside the 10 minute window. **(verify)**